Postman - Budget_App (Backend)

Base URL

`http://localhost:3000`

> **Attention aux adresses :** l'inscription et la connexion sont sur `/register` et `/login`. Toutes les autres routes commencent par `/api/...`.
> (`/api/register` et `/api/login` sont aussi acceptées, ainsi qu'un `/` final : `/api/cards/`.)

1) Créer un compte (POST /register)

Requête Postman

- Method: `POST`
- URL: `http://localhost:3000/register`
- Headers:

- `Content-Type: application/json`

- Body (raw JSON):

```
{
  "fullName": "Test User",
  "email": "test.user1@example.com",
  "password": "Password123!"
}
```

Succès

- `201` avec JSON:

```
{
  "message": "Inscription réussie",
  "userId": 123
}
```

> Si `409`, l'email existe déjà : change `email`.

2) Connexion (POST /login) => récup token

Requête Postman

- Method: `POST`
- URL: `http://localhost:3000/login`
- Headers:

- `Content-Type: application/json`

- Body (raw JSON):

```
{
  "email": "test.user1@example.com",
  "password": "Password123!"
}
```

Tests Postman (onglet Tests)

Récupérer et sauvegarder le JWT :

```
const json = pm.response.json();
if (json && json.token) {
  pm.environment.set('token', json.token);
}
```

Après ce step, `{{token}}` doit avoir une valeur au format `aaa.bbb.ccc`.

3) Tester une API protégée (ex: GET /api/categories)

Requête Postman

- Method: `GET`
- URL: `http://localhost:3000/api/categories`
- Headers (obligatoires):

- `Authorization: Bearer {{token}}`

4) Toutes les routes

Pour toutes les routes `/api/...` (sauf les deux du mot de passe oublié) :

- header `Authorization: Bearer {{token}}` ;
- header `Content-Type: application/json` pour les `POST` / `PUT` ;
- `:id` = le numéro renvoyé à la création (ex : `/api/cards/3`).

Accueil et solde

Méthode	URL	Body (JSON)
GET	`/api/home`	—
GET	`/api/balance`	—
POST	`/api/topup`	`{ "amount": 100 }`

Transactions

Méthode	URL	Body (JSON)			
GET	`/api/transactions`	— (filtres possibles : `?type=income`, `?period=today\	week\	month\	year`, `?category_id=2`)
POST	`/api/transactions`	`{ "title": "Courses", "amount": 25, "type": "expense", "category_id": 2 }` (`category_id` facultatif)			
GET	`/api/transactions/:id`	—			
PUT	`/api/transactions/:id`	`{ "title": "Courses", "amount": 30, "type": "expense" }`			
DELETE	`/api/transactions/:id`	—			

Catégories

Méthode	URL	Body (JSON)
GET	`/api/categories`	—
POST	`/api/categories`	`{ "name": "Loisirs" }`
GET	`/api/categories/:id`	—
PUT	`/api/categories/:id`	`{ "name": "Sorties" }`
DELETE	`/api/categories/:id`	— (refusé si des transactions l'utilisent)

Cartes

Méthode	URL	Body (JSON)
GET	`/api/cards`	—
POST	`/api/cards`	`{ "cardNumber": "4111111111111111", "cardHolderName": "Test User", "expiryDate": "2028-03" }`
GET	`/api/cards/:id`	—
PUT	`/api/cards/:id`	`{ "cardHolderName": "Test User", "expiryDate": "2029-01" }`
DELETE	`/api/cards/:id`	—

Contacts (bénéficiaires)

Méthode	URL	Body (JSON)
GET	`/api/recipients`	—
POST	`/api/recipients`	`{ "name": "Alex Tran", "identifier": "FR76 1234 5678", "type": "bank" }` (`type` : `bank` ou `phone`)
GET	`/api/recipients/:id`	—
PUT	`/api/recipients/:id`	`{ "name": "Alex T.", "identifier": "FR76 1234 5678", "type": "bank" }`
DELETE	`/api/recipients/:id`	—

Transferts et factures

Méthode	URL	Body (JSON)
POST	`/api/transfers/p2p`	Vers un contact : `{ "recipient_id": 3, "amount": 15, "title": "Resto" }`
POST	`/api/transfers/p2p`	Vers un compte saisi : `{ "recipient*name": "Bob", "recipient*identifier": "FR76 9999", "amount": 15 }`
GET	`/api/services`	— (liste des services et fournisseurs)
POST	`/api/services/pay/:id`	`{ "amount": 70, "provider": "EDF" }` (ex : `/api/services/pay/3` = électricité)

> Solde trop bas : `400` `{ "message": "Solde insuffisant" }`.

Profil et mots de passe

Méthode	URL	Body (JSON)
GET	`/api/users/profile`	—
PUT	`/api/users/profile`	`{ "fullName": "Test User", "phone": "0612345678", "ville": "Nantes" }`
POST	`/api/users/change-password`	`{ "oldPassword": "Password123!", "newPassword": "Nouveau123" }`
POST	`/api/users/reset-password`	**Sans token.** `{ "email": "test.user1@example.com" }`
POST	`/api/users/reset-password-confirm`	**Sans token.** `{ "email": "test.user1@example.com", "code": "123456", "newPassword": "Nouveau123" }`

> Le code à 6 chiffres n'est pas envoyé par email : il apparaît dans la console du serveur et dans `logs/app-<date>.log`.

Statistiques

Méthode	URL		
GET	`/api/stats?period=week\	month\	year`
GET	`/api/stats/monthly?months=6`		
GET	`/api/stats/pie?type=expense`		
GET	`/api/stats/line`		
GET	`/api/stats/rewards`		
GET	`/api/users` (compte **admin** seulement, sinon `403`)		

Points importants (pour éviter "jwt malformed")

- `{{token}}` doit être **non vide** et **provenant de /login**.
- Si `/login` échoue, la réponse ne contient pas `token` => Postman garde un token incorrect/ancien.
- Toujours définir `Content-Type: application/json` sur les `POST /register`, `POST /login`, `POST/PUT /api/*`.

Messages d'erreur fréquents

Réponse	Cause	Solution
`Endpoint non trouvé : POST /api/xxx`	L'adresse ou la méthode n'existe pas.	Comparer avec le tableau ci-dessus (le message rappelle ce qui a été envoyé).
`Token manquant` (401)	Header `Authorization` absent.	Ajouter `Authorization: Bearer {{token}}`.
`Token invalide ou expiré` + `invalid signature` (401)	Le token a été créé avec une autre clé (`JWT_SECRET` a changé dans `.env`) ou date de plus de 7 jours.	Refaire `POST /login` pour obtenir un nouveau token.
`Identifiants invalides` (401)	Mauvais email ou mot de passe.	Vérifier le compte, ou en recréer un avec `/register`.